



系统设计学习总结

共 25 个案例

案例名	Requirement	NFR	Entity	Components	API	Object1	Object2
Design Bit.ly	generate shortened URL access original URL by using shortened URL	uniqueness(no 2 long URLs to one shortened) availability latency scalability	Orginal URL Shortened URL User	302 redirect - more control, browser can not cache, click statistics Unique Counter + Base62 Encoding Cache (key-value, short-original) CDN Postgres - DB replication, DB backup scale primary server Unique Counter - batch read	POST /urls { original_URL custom_alias expiration_date } GET /shortURL - 302 tempority redirect original URL	Urls id original_url short_url custom_alias expiration_date creator	-
Design YouTube	upload watch	availability latency large files/resumable scalability	user video video metadata chunk	Block Storage: S3 with presigned URL VideoMetadata DB Message Queue: Transcode/audit process (DAG) CDN Adaptive bitrate streaming Chunk-based upload Redis cache: Video metadata Upload Monitor	POST /upload GET /videos/{videoId}	VideoMetadata id title description user S3 URLs chunks	-
Design a Job Scheduler	schedule jobs to execute (once or recurring) monitor the status	availability latency scalability at least once delivery	user jobs tasks schedule	Message Queue Task executor/coordinator/worker Scheduler/query service Idempotent processing SQS: Visibility timeout mechanism	POST /jobs { task schedule parameters } GET /jobs?userId&status	Job id task schedule parameter userid	Execution id jobId userId status started attempt
Design Yelp	search businesses view business details write/read reviews	availability latency scalability	User Business Review	Elasticsearch: Full-text + geospatial search with CDC Redis Cache Rating aggregation: Optimistic locking (review_num as version) DB constraint: One user one review per business Pre-defined locations: geo_shape in Elasticsearch	GET /business?query&location&category&page GET /business/{businessId} GET /business/{businessId}/reviews POST /business/{businessId}/reviews { rating text } }	Business id lat/long name address description category review_num avgRating locations	Review id businessId userId rating comments created Location id name type points
Design Ticketmaster	search events view event details book tickets	availability for search strong consistency for booking latency scalability	Event Venue Performer Ticket Booking	Elasticsearch: Full-text search Redis: Distributed lock (TTL) + Cache Message Queue: Virtual waiting list WebSockets: Real-time queue updates CDN + Load Balancer	GET /events/{eventId} GET /events?query&start_date&end_date&pageSize POST /bookings/{eventId} { ticketIds paymentDetails } }	Event venue description performer date tickets Ticket eventId seat price status userId	Venue id location seatMap Performer id name description Booking id userId tickets
Design LeetCode	search/view/code problems submit solutions competition leaderboard	availability isolation (sandboxed execution) latency scalability	User Problem Submission Leaderboard	Container/Executor: Sandboxed code execution Redis: Leaderboard (sorted set) Message Queue: Async submission processing Executor coordinator Test case serialization: Multi-language support	GET /problems?page GET /problems/{problemId}?language POST /problems/{problemId}/submit { code language } GET /leaderboard/{competitionId}?page	Problem id title description code stubs test cases	Submission id submitter submitTime language code test result errors



案例名	Requirement	NFR	Entity	Components	API	Object1	Object2
Design Strava	create/pause/resume route (run/ride) record and view activity data view friends' activities	availability latency (real-time accuracy) reliability (offline support) scalability	User Activity Route Friends	Local storage: In-memory + file-based Local DB: Offline support Activity cache Time-based sharding Data tiering: Hot (cache) → Warm → Cold (S3) Long polling: Real-time updates Redis leaderboard: Sorted set	POST /activities { type } PATCH /activities/{activityId} { state } POST /activities/{activityId}/routes { location } GET /activities?mode&starttime&page GET /activities/{activityId}	Activity id type userId createdAt routes status statusEvent distance	Route activityId lat/long timestamp Friend user1 user2 createdAt
Design Tinder	create profile with preferences search potential matches by distance swipe yes/no match notification	strong consistency on match availability for search latency scalability avoid showing previously swiped users	Profile Swipe Match	Elasticsearch: Geospatial search with CDC Push notifications: APNS/FCM Redis + LUA script: Atomic swipe + match check Cassandra: Swipe history with PK (from_user, to_user) for single-partition transactions Cache/Bloom filter: Avoid showing previous swipes Pre-computation: Candidate caching	POST /profile { age_min, age_max distance interestedIn } GET /feed?lat&long POST /swipe/{userId} { decision }	Profile id userId name description agePrefer genderPrefer maxDistance	Match userpair user1 user2 like createdAt
Design a Local Delivery Service	query items by location/delivery time place order	availability for search strong consistency for ordering latency scalability	Item Inventory Distribution Center Order/OrderItem	Elasticsearch: Product search with CDC Availability Service: Check inventory + delivery time Nearby Service: Distance calculation PostgreSQL transaction: ACID guarantees Two-phase filtering: Distance → Travel time Regional partitioning + caching	GET /availability?location&keyword&page POST /order { lat, long items }	DistributionCenter name lat/long Inventory id itemId quantity DCId	Item id name description Order id userId InventoryId quantity
Design Web Crawler	crawl from seed URLs extract and process text data	fault-tolerant politeness (respect robots.txt) efficiency scalability	URLs Metadata Pages	Frontier Queue: SQS with DLQ + exponential backoff S3: Page storage Parsing Queue Rate limiter: Redis + robots.txt DNS caching: Multiple providers Bloom filter: URL deduplication Webpage rendering: JavaScript execution URL Scheduler	N/A	URL id url S3Link lastCrawlTime hash depth	Domain domain lastCrawlTime robots
Design Uber	get fare estimation request a ride match driver accept/decline request	availability for search low latency strong consistency for matching scalability	Rider Driver Ride Fare Location	Matching Service Mapping Service: Fare calculation Notification Service: APN/FCM Location tracking: Redis with geohashing + adaptive update frequency Distributed lock: Redis with TTL Matching queue Geo-sharding	POST /fare { pickupLocation destination } POST /rides { fareId } POST /drivers/location { lat, long } PATCH /rides/{rideId} { accept/reject }	Fare id userId pickup/destination price eta Driver id name rating location vehicle	Ride id fareId driverId pickup/destination status
Design FB Live Comments	post comments view live and historical comments	availability low latency scalability	User Video Comment	SSE (Server-Sent Events): Real-time messaging Zookeeper + Consistent Hashing: Connection routing Dispatcher Cursor pagination	POST /comments/{liveVideoId} { message } GET /comments/{liveVideoId}?cursor&page&sort	Comment id liveVideoId userId createdAt content	-
Design Ad Click Aggregator	redirect ad clicks to advertiser website query ad click metrics	scalability latency fault tolerance/accuracy idempotent processing	Ad click data Metrics	Ad placement service: 302 server-side redirect Click processor Stream: Kinesis/Kafka → Flink Hot ads: AdID partitioning (0-N) Stream retention: Recovery for Flink downtime Flink checkpointing Reconciliation: Raw data in S3 Deduplication: ImpressionID as idempotent key Pre-aggregation: OLAP database	N/A	Advertisement adId redirectUrl metadata impressionID Aggregated Data adId advertiserId (PK) click count minute (SK)	-



案例名	Requirement	NFR	Entity	Components	API	Object1	Object2
Design YouTube Top K	view top-K videos for time windows (1 hour, day, week, month, all time)	latency: 1 min delay after view precision: No approximate algorithms massive views/videos query: < 100ms economical	Video View Time Window	Kafka (Stream) Top-K Heap + Count + Snapshot (Blob): Recovery support Consistent Hashing + Zookeeper: Video ID routing Top-K service Start/Tear-down handling: Delay events, increase start count, decrease tear-down count Kafka retention: 1 month, 2K records for top-K Top-K cache: 1 min TTL	GET /views/top?window&k	-	-
Design a News Aggregator	view aggregated feed from thousands of sources infinite scroll click article to redirect	availability latency scalability	User Source/Publisher Article	Collection Service: RSS feed parsing S3: Thumbnail storage Feed Service Cursor-based pagination: Article ID (ULID/Snowflake) Redis sorted set: Region-based feeds with CDC Intelligent scheduler: Fingerprint-based new content detection Web scraping + RSS + Publisher webhooks with fallback polling S3 + CDN Auto-scaling + cache replicas	GET /feed?page®ion	Article id title summary thumbnailUrl publishDate url hash	Publisher id name url rssFeedUrl lastScraped priority scrapedType
Design a Price Tracking Service	view price history for Amazon products subscribe to price drop notifications	availability latency scalability notification within 1 hour of price drop	Product Price history User Subscription	Web crawler: Chrome for unviewed products, discover new products, verify suspicious data Price history service Subscription service Notification Cron Chrome extension: User-triggered price updates CDC → Kafka: Price change stream → notification workers (or dual write) TimescaleDB: Time-series extension for PostgreSQL	GET /products/{productId}/price?period&granularity POST /subscriptions { productId priceThreshold notificationType }	Product id amazonId lastScraped Price id productId timestamp price	Subscription id userId productId priceThreshold notificationType
Design Online Auction	create auction bid (accept when higher than current highest) view bid history	strong consistency for bidding fault-tolerant, durable low latency for viewing auction bids scalability	User Auction Item Bid	Auction service + Bid service Optimistic locking: maxBidPrice as version in UPDATE WHERE clause Durable message queue: Bid processing SSE: Real-time updates for auction participants (pub/sub) SQS: Visible timeout mechanism Push notifications: APN/FCM	POST /auctions { item, start, end startPrice } POST /auctions/{auctionId}/bids { bid } GET /auctions/{auctionId}	Auction id itemId startTime/endTime startPrice bidPrice status Item id name description imageUrl	Bid id auctionId price userId status createdAt
Design WhatsApp	create chat group send/receive messages offline message delivery media sharing	availability guaranteed deliverability low latency scalability minimize centralized message storage	Users Chats Messages Clients	WebSockets + L4 Load Balancer Chat server GSI (participant): PK = chatId Message Inbox: Per-client delivery queue Delivery flow: Send → ACK → Delete from inbox S3/Block storage: Media files Pub/Sub: ChatId-based (at-most-once OK) Inbox-based on client	WebSocket Messages: createChat sendMessage createAttachment add/removeChatParticipants ack chatUpdate newMessage	Chat id name metadata Participant chatId participantId	Message id user chatId sentAt content Inbox recipientClientId messageId Client id userId type
Design Instagram	create posts (photos, videos) follow other users view chronological feed	availability low latency scalability	User Post Follow Media	S3/Block storage: Presigned URLs Fanout: On-read (cache, celebrity) vs on-write (precompute) Redis: Precomputed feed cache (sorted set) Redis Sentinel: HA AOF/RDB snapshots: Fault tolerance Chunked multipart upload S3 notifications: Server-side image processing CDN + Dynamic Media Optimization	POST /posts { media (photo/video) caption } POST /follows { followerId } GET /feed?cursor&page	Post id (SK) media S3Link creator (PK) timestamp caption	Followers userId (PK) followedId (SK)



案例名	Requirement	NFR	Entity	Components	API	Object1	Object2
Design FB Post Search	create and like posts search posts by keywords sort by recency or like count	availability low latency scalability new posts searchable < 1 min all posts must be discoverable	Post Like User	Ingestion Service: Inverted index creation Search Service Creation Index: List structure Like Index: Sorted set CDN + Redis: Search result caching Bigrams + Count-Min Sketch Kafka + Multiple ingestion services: Keyword-based index sharding Two-phase like indexing: Exponential write, keep 2*N, fresh data from Like Service Storage tiering: Hot (Redis) + Cold (S3)	POST /posts POST /posts/{postId}/like GET /search?query&sort_order (time, likes)	Index Structure: keyword → postIds keyword → sortedSet(postId, likes)	-
Design Robinhood	view live stock prices manage orders (create/cancel)	strong consistency for orders low latency scalability efficiency	Symbol Order User	Symbol Price Processor + Cache + SSE Pub/Sub: Symbol-based subscriptions Order Service + Order Gateway + Trade Processor External Order Metadata DB Order consistency: Status tracking + cleanup job	GET /symbols/{name} POST /orders { position: buy sell symbol quantity priceInCents } DELETE /orders/{orderId} GET /orders	Order id userId (partition key) symbol position quantity price state createdAt externalOrderId	-
Design Google Docs	create document multiple users edit same document view each other's changes see cursor positions	eventual consistency low latency scalability < 100 concurrent editors document durability on server restart	User Document Edit Cursor	WebSockets Doc metadata service Document service: S3/block storage OT (Operational Transformation) or CRDTs (Conflict-free Replicated Data Types) Init flow: Read all Ops, WebSocket sends all applied Ops (Edits, not full doc) Client-side OT Cursor info: In-memory in doc service, sync via WebSocket Zookeeper + Consistent Hashing: Route users to same doc server per document ID Periodic snapshot/compaction: Optimize operation history	POST /docs { title } WebSocket: insert update delete updateCursor	Document id title Operations timestamp docId data	-
Design Dropbox	upload file from any device download file auto-sync across devices share files with others	availability latency (upload, download, sync) scalability large file support security and reliability	File FileMetadata Chunk User	S3 + File Metadata DB: S3 Notifications CDN: Download optimization Sync (Local → Remote): OS-specific events, local queue, last-write-wins Sync (Remote → Local): Polling (stale) + WebSocket/SSE (fresh) Chunked multipart upload + fingerprinting Smart compression Security: Transit (HTTPS), rest (S3 encryption), access control (shareList) Presigned URLs: User access with expiration	POST /files { file fileMetadata } GET /files/{fileId} POST /files/{fileId}/share GET /files/{fileId}/changes POST /files/presigned-url	FileMetadata id name size type userId status S3URL chunks	SharedFile userId fileId
Design a Distributed Cache	get, set, delete operations TTL (Time-To-Live) LRU or LFU eviction	availability latency (< 10ms) scalability (1TB data, 100k QPS)	Key-values	HashMap + Doubly linked list: LRU implementation Cleanup process (Janitor): TTL enforcement HA options: Leader + async replicas (simple) Peer-to-peer/gossip (maximum availability) Sharding + Consistent hashing: Scalability Hot key handling: Write: Key+salt, write batching Read: Replicas/multiple copies Connection pooling	POST /{key} { value } GET /{key} DELETE /{key}	-	-



案例名	 Requirement	 NFR	 Entity	 Components	 API	 Object1	 Object2
Design a Payment System	initiate payment request pay with credit/debit card view payment status	high security strong consistency durability (no data loss) scalability	Payment Merchant Transaction	Payment Service + Transaction Service Payment Gateway 1:N relationship: Payment (intention) → Transactions (attempts) Status flow: created → processing → succeeded/failed Webhooks: Server-to-server event notifications Security: Auth: API Key, request signing, pub/private keys PII protection: iframe isolation, JS SDK immediate encryption, HTTPS, non-reversible tokens Consistency: Internal: Payment + Transaction in RDBMS/ACID External: Event sourcing + reconciliation, rebuild from event log Storage: Cold storage (S3), read replicas Sharding: Merchant ID or time-based	<pre>POST /payments { amountInCents currency description } POST /payments/{paymentId}/transactions { type: payment card: {...} } POST /{merchant_webhook_url} { eventType, paymentId, status }</pre>	Payment id user merchantId amount currency description status	Transaction id paymentId amount type status payment network Merchant id name API key metadata